POLITÉCNICO DE LEIRIA | ESCOLA SUPERIOR DE TECNOLOGIA E GESTÃO

Undergraduate in Games and Multimedia | 2025/2026

Advanced Programming Topics & Computer Graphics

REPORT II

Project2-Xenon Clone Game Engine

Students

Dinis Stasyuk | 2230673

Class Diagrams

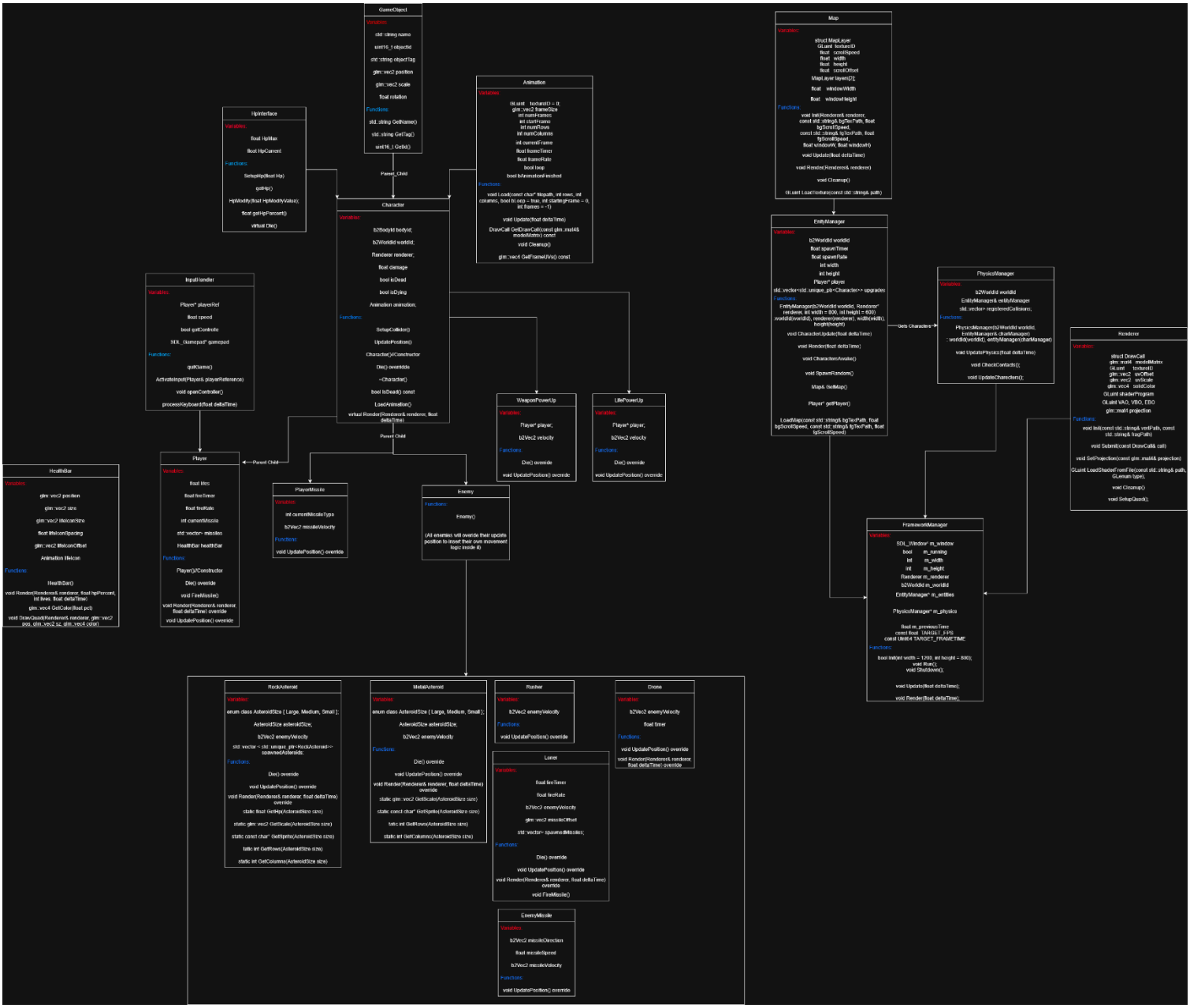


Figure 1-Class diagram of the Xenon clone with Game Engine, available in the files with full resolution

FrameWorkManager

FrameWorkManager is responsible for uniting all the major scripts and managing all game content to produce the game, it runs in the main the following way:

```
#pragma once
#include <ctime>
#include <iostream>
#include "FrameWorkManager.h"

int main()
{
    FrameworkManager framework;
    //Setup and initialazation provide with window size
    if (!framework.Init(1200, 800)) return -1;

    //Loop
    framework.Run();
    //Clear and end
    framework.Shutdown();

    return 0;
}
```

First the manager initializes a window with size the provided, also gets a controller, it will run all code to verify if none of the external libraries cause any issues, creates Renderer, setups Box2d, EnityManager, PhysicsManager and sets run to true. (Will go in further detail later on for the EntityManager and PhysicsManager.)

Second it executes Run() which executes every frame while running it manages FPS capping them, deltaTime providing it to Update and Render and checks if player did not die or for an sdlevent Quit.

Update runs the respective updates from PhysicsManager and EnityManager which to the following:

PhysicsManager -PlayerInput → physics step → position sync box to sprite→ checks contacts

EntityManager- Checks if needs to Spawn → out-of-bounds KILL → add missiles from player → despawns

Render it clears, than it runs Renderer inside the EntityManager so every “entity” gets rendered and gets m_renderer.Present() necessary for this sdl implementation.

With the rendering, entities and physics separated the order of execution for the gameEngine should be the following first a “Awake” start the player and map, player being necessary for the PhysicsManager, run loop with fps, UpdatePhysics, UpdateEntities and than Updating Graphics.

EntityManager

Note that Character is the pawn class which in this current project most objects inherit from.

EntityManager is responsible for holding and spawning every entity, which include enemy, player, missiles, map, rendering it deleting it on death and providing it to other classes that might need it such as the PhysicsManager which requires all characters.

First we run the AwakePlayer() which creates player and stores it than we create the map, we need the player to exist before the physics manager starts so it is the first object created.

We have than the CharacterUpdate, update first updates map, checks with the timer if it is time to spawn a random Enemy, kills any Character outside of the map, player won't ever have this problem since they have a position clamp. Adds first enemies spawn from outside the EntityManager and than it adds any player missiles to the character list so they can also be Updated and Rendered, it than removes from storage any characters that are dead unless it's the player and at last Render inside the EnityManager is triggered.

Render triggers render first inside map than inside all other characters, in the current implementation each object runs their "own" rendering, (they will just utilize the rendering class for rendering but some classes due it in their own way so for safety it triggered this way).

SpawnRandom gets a random location within both width and height and spawns enemies, based on a random roll, providing the enemies with the wordl id and sldRenderer as well as position accordingly. Note that asteroids spawn in different sizes.

EntityManager has also a reference to input handler but only activates it this inpuHandler will be provided to the PhysicsManager to update physics, entities and rendering at separate instances, ensuring consistency.

```
void EntityManager::SpawnRandom()
{
    int spawnX = width;
    int spawnY = height;

    int randomSpawnX = 50.0f + (std::rand() % (int)width);
    int randomSpawnY = (std::rand() % (int)height) - 50.0f;

    int roll = std::rand() % 10;

    std::cout << "SpawnRandom roll: " << roll << std::endl;

    if (roll == 0) characters.push_back(std::make_unique<Rusher>(worldId, glm::vec2(spawnX, randomSpawnY)));
    else if (roll == 1) characters.push_back(std::make_unique<RockAsteroid>(worldId, AsteroidSize::Large, glm::vec2(spawnX, randomSpawnY)));
    else if (roll == 2) characters.push_back(std::make_unique<RockAsteroid>(worldId, AsteroidSize::Medium, glm::vec2(spawnX, randomSpawnY)));
    else if (roll == 3) characters.push_back(std::make_unique<MetalAsteroid>(worldId, AsteroidSize::Large, glm::vec2(spawnX, randomSpawnY)));
    else if (roll == 4) characters.push_back(std::make_unique<MetalAsteroid>(worldId, AsteroidSize::Medium, glm::vec2(spawnX, randomSpawnY)));
    else if (roll == 5) characters.push_back(std::make_unique<MetalAsteroid>(worldId, AsteroidSize::Small, glm::vec2(spawnX, randomSpawnY)));
    else if (roll == 6) characters.push_back(std::make_unique<Loner>(player, worldId, glm::vec2(randomSpawnX, spawnY))); //Only one who spawns from top
    else if (roll == 7) {
        //Drone spawn inside update
        dronesLeft = 8;
        spawnTimer = 0.0f;
        waveSpawnPos = glm::vec2(spawnX, randomSpawnY);
    }
    else if (roll == 8) characters.push_back(std::make_unique<WeaponPowerUp>(player, worldId, glm::vec2(spawnX, randomSpawnY)));
    else if (roll == 9) characters.push_back(std::make_unique<LifePowerUp>(player, worldId, glm::vec2(spawnX, randomSpawnY)));
}
```

```
// ---------------------Enemy spawn timer-----------------------------
spawnTimer += deltaTime;
if (spawnTimer >= spawnRate) {
    spawnTimer = 0.0f;
    SpawnRandom();
}

//Drones spawn one at a time after the wave is triggered in SpawnRandom
if (dronesLeft > 0 && spawnTimer >= 0.4f) {
    spawnTimer = 0.0f;
    dronesLeft--;
    characters.push_back(std::make_unique<Drone>(worldId, waveSpawnPos));
}
```

PhysicsManager

PhysicsManager requires both EntityManager and worldId on construct, it has the following functions:

UpdateCharacter- its simple it gets the characters from EntityManager and updates off their positions the UpdatePosition function is inside all members of Character class and it updates physics inside the characters.

Check Contacts -First it gets all sensor events, clears the vector registeredCollisions, since event collision registers each collision twice with different order and we want to consider each collision only once.

Next, we check every collision event and verify if the pair of objects as been already registered for collision if yes move on to the next collision pair else. Perform a cast to character on both of these objects since we only character objects in this project to check for collisions. Next we check if any of these object is dead or dying since we won't need consider their collisions (tough collisions should be already deactivated if the character is dead or dying, we just confirm it).

Next we get the tags from both objects if tags are different, we proceed and if we verify that one of the objects has the tag "Enemy" and the other has the tag "Player" both objects damage each other, with HpModify from HealthInterface, in which case means that Player and PlayerMissiles, damage Enemies and vise-versa. If the collision wasn't between enemy and Player we check if collision was between player and powerup we cast to player and check the powerup trough tag, if is a collision between player and powerup we trigger Die event in the powerUp which triggers the effect of the PowerUp.

Lastly we register the collision in registeredCollisions independently of events.

The update was already mentioned previously runs this way in scripts - entityManager.input.processKeyboard(deltaTime); → b2World_Step(worldId, 1.0f / 60.0f, 4); → UpdateCharecters();→ CheckContacts();.

```
void PhysicsManager::CheckContacts()
{
    //Get all  SensorEvents tried with GetContactEvents but did not work
    b2SensorEvents events = b2World_GetSensorEvents(worldId);

    //Empties callisions each frame
    registeredCollisions.clear();

    //Check for all start of collisions
    for (int i = 0; i < events.beginCount; ++i)
    {
        //Provides both of the bodies that collide
        b2SensorBeginTouchEvent event = events.beginEvents[i];

        // Sensor events give you sensorShapeId and visitorShapeId
        void* sensorData = b2Body_GetUserData(b2Shape_GetBody(event.sensorShapeId));
        void* visitorData = b2Body_GetUserData(b2Shape_GetBody(event.visitorShapeId));


        //First check if both objects are Character since in this part only collision between characters matter
        Character* objectA = dynamic_cast<Character*>((Character*)sensorData);
        Character* objectB = dynamic_cast<Character*>((Character*)visitorData);


        //Verify first if both objects are Characters since both enemies and players are and we only need to check for them
        if (!objectA || !objectB) continue;

        //verify if collision isn't verified a second time in the same frame just reverse order
        bool alreadyRegistered = false;
        for (auto& [a, b] : registeredCollisions) {
            if ((a == objectA && b == objectB) || (a == objectB && b == objectA)) {
                alreadyRegistered = true;
                break; //Can't to continue here it will just break this for but not the next one
            }
        }
        if (alreadyRegistered) continue;

        //Confirm if the object isn't dead or dying
        if (objectA->isDying || objectA->isDead) continue;
        if (objectB->isDying || objectB->isDead) continue;

        std::string tagA = objectA->getTag();
        std::string tagB = objectB->getTag();

        //If tags are == there is no interaction since objects with the same tag don't interact with each other
        if (tagA == tagB) continue;

        //We verify if the enemy/enemyMissile enters in contact with player/playerMissile
        if ((tagA == "Player" || tagB == "Player") && (tagA == "Enemy" || tagB == "Enemy"))
        {
            //Yes both objects damage each other on contact the values are programmed in a way so missiles always get destroyed on contact, while player destroys anything they touch
            objectA->HpModify(-objectB->damage);
            objectB->HpModify(-objectA->damage);
            std::cout << "Object With name " << objectA->getName() << " with tag " << objectA->getTag() << " has " << objectA->getHp() << " Hp left" << std::endl;
            std::cout << "Object With name " << objectB->getName() << " with tag " << objectB->getTag() << " has " << objectB->getHp() << " Hp left" << std::endl;
        }

        //Need to cast to player since I dont want to obtain powerUp on contact with playerMissile which also has the tag "Player" the powerUp, also works on Player Children
        Player* playerA = dynamic_cast<Player*>(objectA);
        Player* playerB = dynamic_cast<Player*>(objectB);

        if ((playerA || playerB) && (tagA == "PlayerPowerUp" || tagB == "PlayerPowerUp"))
        {
            //Since powerup spawn here they are given player reference at spawn, would have been better if I just created the powerup class but that ran out
            if (playerA)  objectB->Die();
            else  objectA->Die();
        }

        //Registers collisions
        registeredCollisions.push_back({ objectA, objectB });
    }
}
```

InputHandler

Input Handler is run in the physics as shown previously it requires a reference to the player to run. Compared to the previous scripts it is very simple the only thing that is called outside are the void ActivateInput(Player* playerReference) { playerRef = playerReference; }; and processKeyboard(float deltaTime), process keyboard runs a OpenController function which seeks the first controller, there is no separate logic for controller and keyboard though only the arrows work for controller movement.

There is also the special action which attempts fire the missile on the player and a quit on Escape.

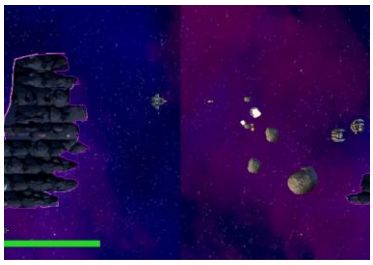
```
void InputHandler::processKeyboard(float deltaTime)
{
    if (!gotController)
    {
        openController();
        gotController = true;
    }
    if (!playerRef) return;

    b2Vec2 playerVelocity = { 0.0f, 0.0f };
    const bool* keyState = SDL_GetKeyboardState(NULL);

    //Only Dpad movement for this part of the project
    if (keyState[SDL_SCANCODE_W] || SDL_GetGamepadButton(gamepad, SDL_GAMEPAD_BUTTON_DPAD_UP))playerVelocity.y = -speed;
    if (keyState[SDL_SCANCODE_S] || SDL_GetGamepadButton(gamepad, SDL_GAMEPAD_BUTTON_DPAD_DOWN))playerVelocity.y = speed;
    if (keyState[SDL_SCANCODE_A] || SDL_GetGamepadButton(gamepad, SDL_GAMEPAD_BUTTON_DPAD_LEFT))playerVelocity.x = -speed;
    if (keyState[SDL_SCANCODE_D] || SDL_GetGamepadButton(gamepad, SDL_GAMEPAD_BUTTON_DPAD_RIGHT))playerVelocity.x = speed;

    //Adding player movement trough box 2D since we will have to account phisics into our movement calcs
    b2Body_SetLinearVelocity(playerRef->bodyId, playerVelocity);

    //Keys for special action
    if (keyState[SDL_SCANCODE_SPACE] || keyState[SDL_SCANCODE_LSHIFT] || keyState[SDL_SCANCODE_RSHIFT]
        || SDL_GetGamepadButton(gamepad, SDL_GAMEPAD_BUTTON_LEFT_SHOULDER) || SDL_GetGamepadButton(gamepad, SDL_GAMEPAD_BUTTON_RIGHT_SHOULDER)
        || SDL_GetGamepadAxis(gamepad, SDL_GAMEPAD_AXIS_LEFT_TRIGGER) || SDL_GetGamepadAxis(gamepad, SDL_GAMEPAD_AXIS_RIGHT_TRIGGER))
    {
        playerRef->FireMissile();
    }

    //Quit game
    if (keyState[SDL_SCANCODE_ESCAPE]) { std::cout << "Quit Game" << std::endl; quitGame(); }
}
```

Animation

Imports sprites with file path, number of rows and columns which it uses in its calculation to get the amount of frames and the size of each sprite, there can be set manually the number of frames in the animation if it loops. We don't need to remove magenta when loading since shader does it, and we have to bind textures with openGl as well as set its parameters.

Map

Map has constantly 2 tiles scrolling for each layer, can set scrolling speed of each one manually receives the speed, renderer and both background textures paths. It isn't particularly efficient nor good or well made.



Renderer

At startup the renderer loads the vertex and fragment shader, creates program and setups the squares where the sprites will be drawn.

Render gets the shaders from files, and the fragment shader removes any magenta from the spriteSheets, and the vertex shader just has 2 coordinates with set locations layout(location = 0) in vec2 position layout(location = 1) in vec2 texCoord.

Renderer loads the sprite sheet with rectangles this way

```
void Renderer::SetupQuad()
{
    //This is done in a way to match the box 2D center
    float vertices[] = {
        // pos          // uv
       -0.5f,  0.5f,  0.0f, 1.0f,  // top-left
        0.5f,  0.5f,  1.0f, 1.0f,  // top-right
        0.5f, -0.5f,  1.0f, 0.0f,  // bottom-right
       -0.5f, -0.5f,  0.0f, 0.0f,  // bottom-left
    };
    unsigned int indices[] = { 0, 1, 2,  0, 2, 3 };

    //Generate all open gl necesseties and assign vertices correctly
    glGenVertexArrays(1, &VAO);
    glGenBuffers(1, &VBO);
    glGenBuffers(1, &EBO);

    glBindVertexArray(VAO);
    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices, GL_STATIC_DRAW);

    //My vertex shaders allow for this
    //Position (location 0)
    glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, 4 * sizeof(float), (void*)0);
    glEnableVertexAttribArray(0);
    //Uv (location 1)
    glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, 4 * sizeof(float), (void*)(2 * sizeof(float)));
    glEnableVertexAttribArray(1);

    glBindVertexArray(0);
}
```

Renderer in the submit uses first the shader than uploads matrices assigns texture to have in a way to have it coincide with the box2D position, if there was no texture solid red rectangle is rendered, we bind the texture/solid square and draw the quad in the correct position, with the correct UV coordinates and with the colour from the texture/square.

HealthBar

Draws both lifes and Hp percentage, lifes its just a player sprite sheet that gets repeated based on the amount of lifes while, the HpPercentage are 2 bars one empty that doesn't change another that starts full at green and changes colour as player loses Hp, from green to yellow to red.

HpInterface

The HpInterface is very simple it is provided to all Characters it has 3 getters a Die which is meant to be overrided by the objects that own it and HpModify which modifies hp both for healing and taking damage, it can't be above the MaxHp and if the player Hp reaches =< 0 it triggers the Die event.

GameObject and Character

The main class we have in this project is the GameObject class this class contains the variables, name, objectId, objectTag, position, scale, rotation and is the ancestor for most of the objects of this project and serves as the parent to the main class Character.

Character class is the main class for all objects with, collisions, animation and health and it sets them up with the help of the HpInterface, Animation class and box2d.

Character adds variables and functions for collision, rendering, damage, animation and for dying/dead.

It is obligatory to provide the character with a renderer and a world Id the rest has defaults, also all children of Character need to provide their own spriteSheet for the animation to load.

For collisions a box collider2D sensor is made that follows the sprite and position of the character with the help of UpdatePosition.

Rendering inside Character updates animation, submits draws inside the renderer, and prevents the player from being dead while the death animation is played.

The Die from healthInterface is override so that the characters trigger a death animation and while dying they don't have any colliders as well as stops their movement.

Player

Player inherits from character and adds the fire missile, and adds a clamp to their position based on maps limits which it receives on construct, the player speed is set by the input handler outside of player which runs in physics and player infiltrates the updates of the Renderer to run a timer for missile fireRate and to erase draw the HealthBar.

Player also has intCurrentMissile and lifes these variables are modifiable trough the powerUps, and allow respectively to change the damage and sprite of the Missile and not die when Hp reachs =<0.

Lastly player owns the HealthBar allowing the display of the current number lifes and Hp percentage of the player

PlayerMissile

Just has the update position and a set velocity to go Up, most of the logic involving it is outside of it be it in the CheckContacts or the spawning inside the player. Important that this has the tag "Player" and 1 hp so dies on colliding with any setup object. Damage and sprite change based on the current player missiles.

PlayerPowerups

They are just characters that get a refence to the player, on construct, with a horizontal speed, that override Die to change a value inside the player. Powerups only have a standard death when colliding with the player when they get killed by leaving the map they are killed without triggering death.

LifePowerUp

On Die increments player current missile can be done twice.

WeaponPowerUp

On Die increments player player life and player can’t have more than 3 lifes.

Enemy

Enemies inherit from character, the enemy class initially had more functionality, but gradually through the process of this work it lost many of the things that distinguished it from other characters since its functionality was added to the character class, and at the moment it is used by the only the Loner and Rusher which are both initialized by the SpawnRandom inside EntityManager.

Of all the enemies they have some form of velocity and a Position() override which dictates their movement behaviour now I´ll go on their individual differences.

Rusher- Just has the basic update position and velocity which is faster than any other enemy.

Drone- Its movement is wave like since the vertical velocity is calculated with cos and changes with a timer, Drones spawn in waves which is managed inside the enemy manager, each one of them spawns in the same position just with a small delay which when combined with the movement behaviour gives them the wave like movement.

Loner- Has a basic movement, but what separates them from the rest is that they fire Enemy Missile towards the player, at their spawn they get a player refence and than calculate a shotting direction of the missile to go towards the player. Fires the missiles every 2 seconds this behaviour infiltrates the Renderer since it already gets the deltaTime necessary for the timer, also adds the spawned missiles to a list which is acquired by the enemy manager.

EnemyMissile-Spawned by the Loner is provided the direction to go towards the player at the moment of spawn.

RockAsteroid- Has 3 different sizes Large, Medium and Small that change its, sprite, Hp and scale. Upon death if it ins´t a small asteroid it splits into 2 smaller asteroids with opposites vertical speeds, also adds them to a list list which is acquired by the enemy manager.

MetalAsteroid-Is the same as the RockAsteroid with the major difference of being invicible which it achieves by overriding Die() and leaving it empty, so this asteroid doesn´t split or have Hp difference between sizes.

Conclusion

The things that weren't done the external library for the gameEngine and its implementation in the xenon clone game, player animations for movement, score UI Text, player companion powerup.

Resource management for this project was lacking and messy having parts of the script that became redundant due to others, this happened because I would better the architecture in the first project of Advanced Programmin and then implement it in the second project forgetting to delete some parts.

References

The Cherno. (2019, September 15). *How to Build a 2D Renderer | Game Engine series* [Video]. YouTube. https://www.youtube.com/watch?v=ULO8JhM9FsQ, used many other videos

Lazy Foo' Productions - Beginning Game Programming v2.0. (n.d.). https://lazyfoo.net/tutorials/SDL/

Box2D: Events. (n.d.). https://box2d.org/documentation/group__events.html

Technologies, U. (n.d.-b). *Unity - Manual: Execution Order of event functions*. https://docs.unity3d.com/560/Documentation/Manual/ExecutionOrder.html

Technologies, U. (n.d.-c). *Unity - Manual: Fundamental Unity types*. https://docs.unity3d.com/6000.3/Documentation/Manual/fundamental-unity-types.html

Simple Direct Media Layer. (n.d.). SDL_GamepadButton. SDL Wiki.

https://wiki.libsdl.org/SDL3/SDL_GamepadButton

As well as the materials provided in class, I manually looked through some UE5 directories for some inspiration.